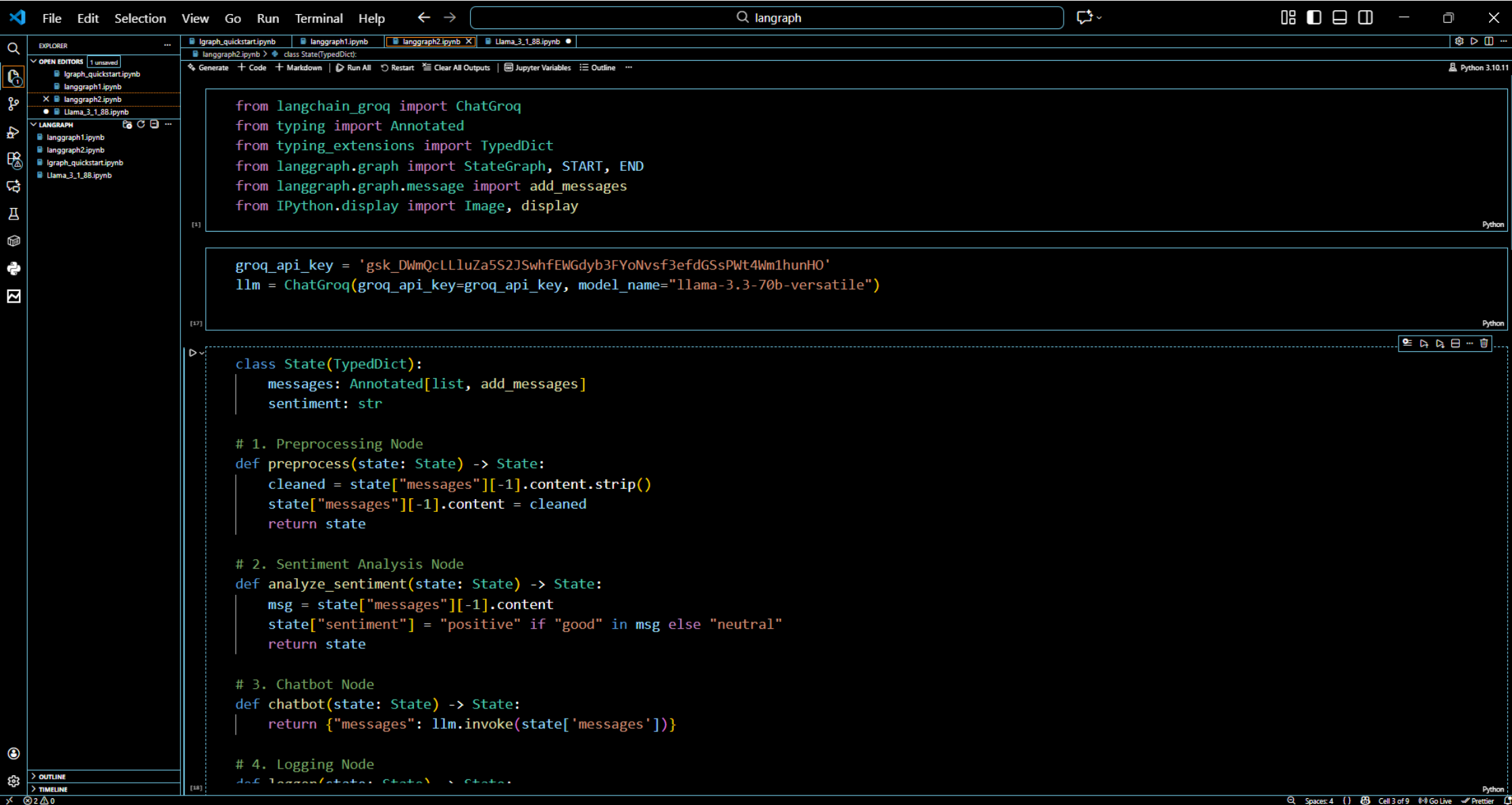




File Edit Selection View Go Run Terminal Help

langraph

Python 3.10.1

Python

EXPLORER

OPEN EDITORS 1 unsaved

lgaph_quickstart.ipynb

langgraph1.ipynb

langgraph2.ipynb

Llama_3_1_8B.ipynb

LANGGRAPH

langgraph1.ipynb

langgraph2.ipynb

lgaph_quickstart.ipynb

Llama_3_1_8B.ipynb

lgaph_quickstart.ipynb

langgraph1.ipynb

langgraph2.ipynb

Llama_3_1_8B.ipynb

Generate + Code + Markdown + Run All + Restart + Clear All Outputs + Jupyter Variables + Outline

4. Logging Node

def logger(state: State) -> State:

print(f"LOG: {state['messages'][-1].content}, Sentiment: {state.get('sentiment')}")

return state

[18]

Python

Build the graph

builder = StateGraph(State)

builder.add_node("preprocess", preprocess)

builder.add_node("analyze_sentiment", analyze_sentiment) # renamed

builder.add_node("chatbot", chatbot)

builder.add_node("logger", logger)

[19]

Python

<langgraph.graph.state.StateGraph at 0x22385f5b970>

...

Define flow

builder.add_edge(START, "preprocess")

builder.add_edge("preprocess", "analyze_sentiment") # renamed

builder.add_edge("analyze_sentiment", "chatbot") # renamed

builder.add_edge("chatbot", "logger")

builder.add_edge("logger", END)

[20]

Python

<langgraph.graph.state.StateGraph at 0x22385f5b970>

...

graph = builder.compile()

[21]

Python

Draw the graph

try:

display(Image(graph.get_graph().draw_mermaid_png()))

except Exception:

pass

[22]

Python

> OUTLINE

> TIMELINE

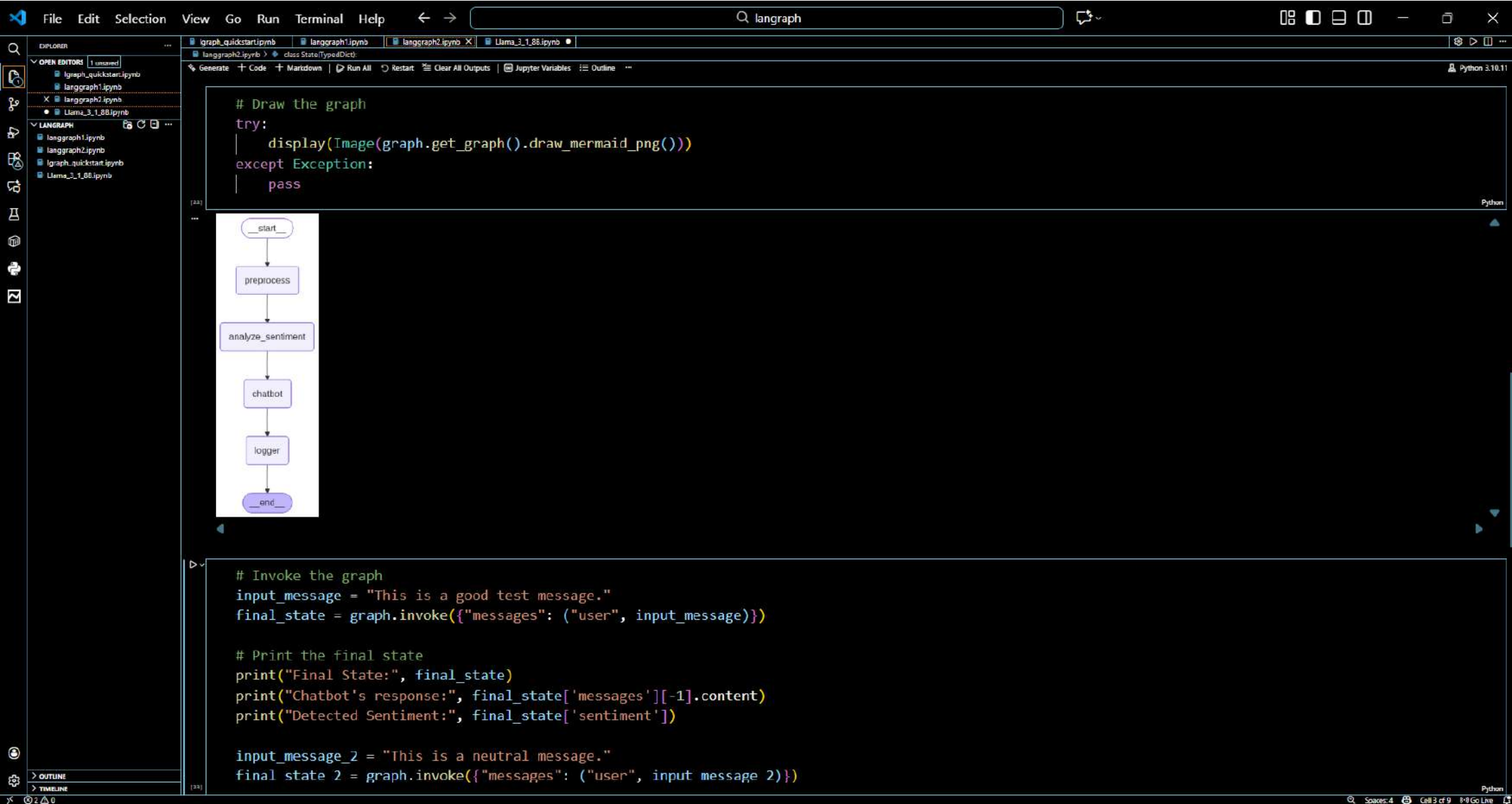
start

Spaces: 4 ()

Cell 3 of 9

Go Live

Prettier



File Edit Selection View Go Run Terminal Help

langraph

EXPLORER

OPEN EDITORS 1 unsaved

langraph2.ipynb

langraph1.ipynb

langraph2.ipynb

Ulama_3_1_88.ipynb

LANGGRAPH

langgraph1.ipynb

langgraph2.ipynb

lgraph_quickstart.ipynb

Ulama_3_1_88.ipynb

lgraph_quickstart.ipynb

langgraph1.ipynb

langgraph2.ipynb X

Ulama_3_1_88.ipynb

Generate

Code

Markdown

Run All

Restart

Clear All Outputs

Jupyter Variables

Outline

Python 3.10.11

graph TD; input([input]) --> end([end]);

```
# Invoke the graph
input_message = "This is a good test message."
final_state = graph.invoke({"messages": ("user", input_message)})

# Print the final state
print("Final State:", final_state)
print("Chatbot's response:", final_state['messages'][-1].content)
print("Detected Sentiment:", final_state['sentiment'])

input_message_2 = "This is a neutral message."
final_state_2 = graph.invoke({"messages": ("user", input_message_2)})
print("Final State 2:", final_state_2)
print("Chatbot's response 2:", final_state_2['messages'][-1].content)
print("Detected Sentiment 2:", final_state_2['sentiment'])
```

[23]

Python

LOG: It looks like your test message has been successfully sent. If you have any actual questions or need assistance, feel free to ask., Sentiment: positive
Final State: {'messages': [HumanMessage(content='This is a good test message.', additional_kwargs={}, response_metadata={}, id='0af5ffd4-60d7-407c-aa5f-f29e54f-...
Chatbot's response: It looks like your test message has been successfully sent. If you have any actual questions or need assistance, feel free to ask.
Detected Sentiment: positive
LOG: This is a response to a neutral message., Sentiment: neutral
Final State 2: {'messages': [HumanMessage(content='This is a neutral message.', additional_kwargs={}, response_metadata={}, id='5363dca3-628f-4fa8-b000-18048a5-...
Chatbot's response 2: This is a response to a neutral message.
Detected Sentiment 2: neutral

1

Python

OUTLINE

TIMELINE

2 0

Spaces: 4

Cell 3 of 9

Go Live